

A Project Report on
**Automated Lifting and Synthesis of Legacy Code
into Tensor IR using MLIR**
for the course CIP67 - MINI PROJECT WORK

Submitted in partial fulfillment of the requirements for the award of the degree of
**Bachelor of Engineering in Computer Science and Engineering
(Artificial Intelligence & Machine Learning)**

By:

Reynesh K	1MS23CI098
Rishab Tarakesh	1MS23CI099
Yashovardhan Singh Rathore	1MS23CI137

Under the Guidance of:

Dr. Mohana Kumara S

Associate Professor (CSE(AI&ML))

Dr. Sini Anna Alex

Associate Professor (CSE(AI&ML))

M S RAMAIAH INSTITUTE OF TECHNOLOGY
(Autonomous Institute, Affiliated to VTU)
BANGALORE-560054
www.msrit.edu
2025-2026

CERTIFICATE

Certified that the Mini Project Work (CIP67) entitled "Automated Lifting and Synthesis of Legacy Code into Tensor IR using MLIR" is a bonafide work carried out by Reynesh K (1MS23CI098), Rishab Tarakesh (1MS23CI099), and Yashovardhan Singh Rathore (1MS23CI137), students of M. S. Ramaiah Institute of Technology, Bengaluru, in partial fulfillment of the requirements for the award of Bachelor of Engineering in Computer Science and Engineering (AI&ML) of Visvesvaraya Technological University, Belagavi during the academic year 2025-26. It is certified that all corrections and suggestions indicated during internal assessment have been incorporated, and the report satisfies the prescribed academic requirements for Mini Project Work.

Signature of Guide 1

Dr. Mohana Kumara S

Associate Professor
(CSE(AI&ML))

Dept. of CSE(AI&ML)

Signature of Guide 2

Dr. Sini Anna Alex

Associate Professor
(CSE(AI&ML))

Dept. of CSE(AI&ML)

Signature of HoD

Dr. Siddesh G M

Professor & Head

Dept. of CSE(AI&ML) and
Dept. of CSE(CS)

External Examiners

The external examiners will be nominated by the university at the time of final viva-voce evaluation.

DECLARATION

We, Reynesh K (1MS23CI098), Rishab Tarakesh (1MS23CI099), and Yashovardhan Singh Rathore (1MS23CI137), hereby declare that the work presented in this mini project report has been carried out by us at M. S. Ramaiah Institute of Technology, Bengaluru, under the guidance of Dr. Mohana Kumara S, Associate Professor (CSE(AI&ML)) and Dr. Sini Anna Alex, Associate Professor (CSE(AI&ML)). This report has not been submitted, either in part or in full, for the award of any diploma or degree of this or any other university.

Signature
Reynesh K

1MS23CI098

Signature
Rishab Tarakesh

1MS23CI099

Signature
Yashovardhan Singh Rathore
1MS23CI137

ACKNOWLEDGEMENT

We take this opportunity to express our sincere gratitude to everyone who supported the successful completion of this mini project. We thank the Management of M. S. Ramaiah Institute of Technology and **Dr. N.V.R Naidu**, Principal, for providing us with a strong academic environment and the facilities required to carry out this work.

We express our heartfelt thanks to **Dr. Siddesh G M**, Head of the Department of Computer Science and Engineering (AI&ML), for continuous encouragement and institutional support.

We are deeply grateful to our guides, **Dr. Mohana Kumara S** and **Dr. Sini Anna Alex**, Associate Professors, Department of Computer Science and Engineering (AI&ML), for their constant mentoring, technical insights, and constructive feedback throughout every stage of the project. Their guidance helped us refine the problem definition, improve implementation quality, and maintain a verification-first approach in our work.

We also thank **Dr. A N Ramya Shree**, Mini Project Coordinator, for timely reviews and valuable suggestions during project evaluations.

Finally, we place on record our gratitude to all teaching and non-teaching staff of the CSE (AI&ML) department, and to our parents and friends for their sustained motivation and support.

Reynesh K (1MS23CI098)

Rishab Tarakesh (1MS23CI099)

Yashovardhan Singh Rathore (1MS23CI137)

Abstract

Legacy scientific and engineering software is predominantly written as scalar loop nests in C, C++, and Fortran, which makes direct use of modern tensor accelerators difficult without expensive manual rewriting. This project implements LoopHole, a verification-driven lifting tool that translates loop-level MLIR Affine programs into higher-level MLIR tensor dialects, primarily Linalg and partially StableHLO. The implemented pipeline consists of deterministic Affine extraction, sketch-based candidate generation, SymPy-guided confidence scoring, formal equivalence checking using Z3, and dialect-specific code emission through a command-line interface. The current implementation supports practical kernels such as matrix multiplication, transpose, dot product, 1D and 2D convolution, elementwise addition, and reduction patterns. In a strict-profile batch run on seven canonical fixture programs, the system produced seven accepted outputs with formal equivalence proofs and generated lifted MLIR artifacts for all processed inputs. The recorded verification times ranged from 5.27 ms to 33.98 ms, with an average of approximately 12.87 ms, indicating that the current proof strategy is efficient for constrained dense kernels. The project also provides reproducible workflows through Dockerized Polygeist and structured batch reports for weekly benchmarking and regression tracking. Overall, this work demonstrates the practical feasibility of automated lifting with verification-aware synthesis in the MLIR ecosystem, while also identifying open challenges in StableHLO parity, dynamic-shape proof strategies, and broader real-world kernel coverage.

Contents

Acknowledgement	i
Abstract	ii
List of Figures	v
List of Tables	vi
1 INTRODUCTION	1
1.1 General Introduction	1
1.2 Problem Statement	1
1.3 Objectives of the project	2
1.4 Project deliverables	2
1.5 Current Scope	2
1.6 Future Scope	3
2 LITERATURE SURVEY	4
2.1 Introduction	4
2.2 Related Works	4
2.3 Conclusion of Survey	5
3 PROJECT REQUIREMENT SPECIFICATIONS	6
3.1 Hardware Requirements	6
3.2 Software Requirements	7
3.3 Functional Requirements	7
3.4 Non-Functional Requirements	8
4 DESIGN	9
4.1 Introduction	9

4.2	Architecture Design	9
4.3	User Interface Design	10
4.4	State Diagram	11
5	IMPLEMENTATION	12
5.1	Pseudocode	12
5.2	Implementation Details	12
6	RESULTS	15
6.1	Result Snapshots	15
6.2	Comparative Analysis	15
6.3	Performance Analysis	16
6.3.1	Applications	17
6.3.2	Limitations	17
7	CONCLUSION & FUTURE WORK	18
7.1	Conclusion	18
7.2	Future work	18
	References	20

List of Figures

6.1	Placeholder for strict-profile batch summary output.	15
6.2	Placeholder for per-kernel verification timeline view.	15
6.3	Placeholder for source-vs-lifted artifact visualization.	15

List of Tables

2.1	Comparison of major lifting frameworks relevant to LoopHole	5
3.1	Hardware requirements for development and validation	6
3.2	Software stack used by the project	7
3.3	Functional requirements for LoopHole	7
3.4	Non-functional requirements and quality targets	8
4.1	Core architecture stages and responsibilities	10
4.2	CLI interface design and operational intent	10
4.3	Operational state flow for one lifting request	11
5.1	Implemented modules and their roles	13
5.2	Operational command workflows used in the project	13
6.1	Comparative positioning against key lifting systems	16
6.2	Strict-profile batch summary (fixture set size = 7)	16
6.3	Per-kernel verification latency from strict-profile report	17

Chapter 1

INTRODUCTION

1.1 General Introduction

Modern computing platforms have shifted from general-purpose CPU scaling to heterogeneous acceleration through GPUs, TPUs, and specialized AI hardware. Legacy numerical software, however, is still largely written as scalar loop nests in C, C++, and Fortran, which hides tensor-level intent from downstream compilers. This mismatch creates a practical barrier: high-value codebases remain difficult to optimize for modern hardware without expensive manual rewriting.

LoopHole addresses this gap by lifting low-level loop programs into high-level MLIR tensor dialects so that mature compiler stacks can optimize them effectively. The project is built around the MLIR ecosystem [1], uses Polygeist-generated Affine IR as the structured loop input [2], and targets Linalg and StableHLO output forms that are consumable by accelerator backends.

1.2 Problem Statement

The core problem is to automatically recover mathematical semantics from scalar loop code and emit equivalent tensor IR with a trust-aware verification signal. In practical terms, given an affine loop nest, the system must identify whether it represents operations such as matrix multiplication, transpose, convolution, elementwise map, or reduction, and then emit an equivalent high-level operation with sufficient confidence and correctness checks.

This problem combines compiler analysis, symbolic reasoning, and formal verification. A pure pattern-matching approach is brittle, while unconstrained synthesis is computationally expensive. Therefore, the project adopts a constrained sketch-guided pipeline with formal checks where feasible, and explicit trust-state reporting for each lift.

1.3 Objectives of the project

1. Build an end-to-end lifting pipeline from MLIR Affine input to Linalg/StableHLO output.
2. Implement deterministic extraction of loop structure, memory access patterns, and arithmetic operations.
3. Maintain a reusable sketch library for canonical tensor operations.
4. Rank candidate sketches using symbolic tracing and confidence scoring.
5. Validate top candidates through Z3-based equivalence checks where applicable.
6. Emit compilable lifted MLIR artifacts and expose the workflow through CLI commands.
7. Provide structured batch reporting to support repeatable benchmarking and regression tracking.

1.4 Project deliverables

The current project delivers the following concrete artifacts:

- A working Python package for lifting and verification-aware synthesis.
- CLI workflows for single-file lift, verification, batch processing, and demo runs.
- Module-level implementation for parsing, sketch matching, formal checking, and emission.
- Docker-supported reproducibility path for Polygeist plus MLIR-based compile-then-lift workflows.
- Automated report generation in JSON format for batch experiments.
- A test suite and project documentation that record implemented behavior, limitations, and improvement tracks.

1.5 Current Scope

The implemented scope focuses on affine loop kernels with clear structured memory access and deterministic loop bounds. Within this scope, the system supports practical dense operations including matrix multiplication, transpose, dot, elementwise map, 1D/2D convolution patterns, and reduction forms. The project also includes strict-profile reporting to separate trusted outcomes from exploratory runs.

1.6 Future Scope

Future scope extends toward stronger proof reliability for weak kernels, broader StableHLO parity, and larger real-world corpus support. Beyond engineering hardening, the research direction includes sparse tensor lifting, transform-dialect scheduling synthesis, and dynamic-shape reasoning, aligned with open areas identified in the project roadmap and literature [6].

Chapter 2

LITERATURE SURVEY

2.1 Introduction

Automated lifting from scalar loops to tensor representations has progressed from handcrafted rewrite rules to synthesis-driven and neuro-symbolic techniques. For this project, the most relevant lineage includes `mlirSynth` [3], `Tenspiler` [4], `Tensorize` [5], and `STAGG` [6]. These systems collectively define the design space for search strategy, verification strategy, scalability, and trust guarantees.

2.2 Related Works

Early MLIR lifting efforts emphasized structural rewrite opportunities, while newer systems use symbolic reasoning and synthesis to generalize beyond fixed hand-coded patterns. `mlirSynth` demonstrated retargetable raising via enumerative synthesis in MLIR and showed strong accelerator potential, but practical search complexity remains a key challenge [3]. `Tenspiler` introduced SMT-backed verification workflows for tensor lifting, highlighting formal correctness benefits as well as solver-timeout boundaries on harder kernels [4]. `Tensorize` improved scalability by combining symbolic tracing with sketch-and-solve decomposition, reporting strong empirical gains over baseline compilation on dense kernels [5]. `STAGG` pushed the field further by combining LLM-guided probabilistic grammars with guided search and formal checks, improving solve rates and synthesis efficiency on benchmark suites [6].

Table 2.1: Comparison of major lifting frameworks relevant to LoopHole

Framework	Year/Venue	Core Method	Strength	Limitation Relevant to this Project
mlirSynth	2023 / PACT	Bottom-up enumerative synthesis in MLIR dialect space	Retargetable lifting and strong TPU gains reported	Exponential search growth for longer operation sequences [3]
Tenspiler	2024 / ECOOP	Rosette plus SMT verification	Formal proof-oriented lifting pipeline	Invariant complexity and SMT timeout pressure on harder kernels [4]
Tensorize	2025 / CGO	Symbolic tracing plus algebraic sketch solving	Better scaling on dense kernels and high benchmark coverage	Primarily dense/static regimes; limited sparse and dynamic-shape support [5]
STAGG	2025 / PLDI	LLM-guided probabilistic grammar plus guided search	High benchmark solve rate with efficient search guidance	Focuses on dense tensor algebra; novelty now requires new open fronts [6]
LoopHole (this project)	2026 / Project POC	Deterministic Affine extraction + sketch ranking + Z3 checks + MLIR emission	Practical end-to-end implementation with trust-state reporting and reproducible workflow	StableHLO parity and broader proof envelope still under active hardening

In summary, the literature indicates that practical systems benefit from combining structural extraction, symbolic guidance, and formal checks rather than relying on a single mechanism. LoopHole adopts this blended strategy and prioritizes implementation-first trust signals for each produced output.

2.3 Conclusion of Survey

The survey shows a clear evolution toward hybrid synthesis architectures that balance correctness, scalability, and usability. Existing state-of-the-art systems establish strong baselines, but open challenges remain in dynamic shapes, sparse representations, and stronger operational trust defaults. These gaps motivate the current project direction and inform the design choices used in the implementation chapters.

Chapter 3

PROJECT REQUIREMENT SPECIFICATIONS

3.1 Hardware Requirements

Table 3.1: Hardware requirements for development and validation

Component	Minimum Requirement	Recommended for Smooth Workflow
Processor	4-core x86-64 CPU	8-core or higher CPU for faster local test and build cycles
Memory (RAM)	8 GB	16 GB or higher, especially for Docker and toolchain workflows
Storage	10 GB free space	25 GB or higher for LLVM/MLIR artifacts and report outputs
Operating System	Windows 10/11, Linux, or macOS	Linux/WSL2 for easier Polygeist/MLIR workflow integration
Network	Standard internet access	Stable broadband for package/toolchain pulls and container images

3.2 Software Requirements

Table 3.2: Software stack used by the project

Software	Version/Type	Purpose
Python	3.10+	Runtime for pipeline modules, CLI, and reporting scripts
z3-solver	4.12+ (Python package)	Formal equivalence checking and trust-state generation [8]
SymPy	1.12+	Symbolic tracing and sketch confidence scoring
LLVM/MLIR toolchain	Compatible build or container image	IR ecosystem, dialect compatibility, and downstream validation [1]
Polygeist	Frontend toolchain	C/C++ to MLIR Affine conversion path [2]
Docker (optional but recommended)	Current stable release	Reproducible compile-then-lift workflow
Pytest	Current stable release	Unit and integration testing automation

3.3 Functional Requirements

Table 3.3: Functional requirements for LoopHole

ID	Requirement	Validation Evidence
FR-1	Parse affine/scf loop-based MLIR and extract loop/memory/computation metadata	Affine extraction module and parser-focused tests
FR-2	Maintain sketch definitions for Linalg and StableHLO target operations	Sketch library and sketch listing CLI path
FR-3	Rank candidate operations using symbolic trace confidence	SymPy tracer and ranked candidate flow
FR-4	Verify candidate equivalence using Z3 where proof path is available	Verification reports with PROVED/UNPROVED/REFUTED states
FR-5	Emit lifted MLIR for accepted outputs in selected target dialect	Linalg/StableHLO emitter modules and generated artifacts
FR-6	Support CLI execution for single-file lift, verify, batch, and demo paths	Implemented loophole command surface
FR-7	Generate structured batch reports for repeatable benchmark runs	JSON report schema and weekly benchmark helper

3.4 Non-Functional Requirements

Table 3.4: Non-functional requirements and quality targets

Quality Attribute	Target	Rationale
Correctness transparency	Explicit trust-state reporting for every lift result	Avoid silent acceptance of uncertain outputs
Reproducibility	Same fixture set should produce stable batch report schema and meta-data	Enables longitudinal benchmarking and regression tracking
Maintainability	Modular code organization by stage (parser, matcher, verifier, emitter, CLI)	Supports targeted debugging and parallel development
Usability	CLI-first workflow with meaningful diagnostics	Enables rapid experimentation for contributors
Portability	Linux/WSL2/Docker-compatible execution paths	Reduces environment-specific setup friction
Performance efficiency	Verification latency suitable for fixture-scale CI runs	Maintains practical iteration speed during development

Chapter 4

DESIGN

4.1 Introduction

The design of LoopHole follows a staged pipeline architecture to keep extraction, reasoning, and emission concerns independent. This separation improves maintainability and supports targeted hardening of weak stages without rewriting the entire system. The design is aligned with MLIR-based lifting workflows and emphasizes explicit trust-state outcomes for each transformed artifact.

4.2 Architecture Design

At a high level, the system processes loop-based MLIR through four major stages: parsing, candidate generation, verification, and emission. The parser builds structured loop metadata; the matcher selects likely operation sketches; the verifier evaluates equivalence on top candidates; and the emitter generates target-dialect MLIR.

Table 4.1: Core architecture stages and responsibilities

Stage	Primary Module	Input/Output	Responsibility
Parsing	<code>affine_extractor.py</code>	Affine MLIR text to LoopNestInfo	Extract loops, bounds, access patterns, and arithmetic structure
Candidate generation	<code>sympy_tracer.py</code> + <code>sketch_library.py</code>	LoopNestInfo to ranked sketches	Build symbolic expressions and score sketch compatibility
Verification	<code>z3_checker.py</code>	Ranked sketch to verification report	Run formal/heuristic equivalence checks and assign trust state
Emission	<code>emitter.py</code>	Accepted sketch to MLIR artifact	Emit Linalg/StableHLO-compatible output text
Orchestration	<code>lifter.py</code>	Stage outputs to final result object	Coordinate policy profile, acceptance logic, and diagnostics

This staged architecture supports both exploratory development and strict-profile evaluation by centralizing acceptance policy in orchestration while retaining modular internals.

4.3 User Interface Design

The primary user interface is command-line based. This choice is intentional: contributors can run reproducible experiments, integrate with scripts, and automate CI-friendly benchmarks without GUI dependencies.

Table 4.2: CLI interface design and operational intent

Command	Purpose	Typical Output
<code>loophole lift</code>	Lift a single MLIR input into selected target dialect	Lifted MLIR, trust state, and diagnostics
<code>loophole verify</code>	Run verification flow without focus on emission	Verification state and proof-related notes
<code>loophole batch</code>	Process multiple fixtures in one run	Structured JSON report, per-file status summary
<code>loophole demo</code>	Execute canonical fixture walkthrough	Quick project health snapshot and sample outputs
<code>loophole sketches</code>	Inspect available sketch definitions	Operation list for both target dialects

The interface returns explicit trust states (PROVED, UNPROVED_TIMEOUT, REFUTED), which supports transparent downstream decision-making.

4.4 State Diagram

The execution can be represented as a deterministic state machine with guarded transitions based on evidence quality and policy profile.

Table 4.3: Operational state flow for one lifting request

State	Transition Condition	Next State
Input Loaded	MLIR source is readable and non-empty	Parsed
Parsed	Loop and access metadata extracted successfully	Candidate Ranked
Candidate Ranked	At least one viable sketch candidate found	Verification Attempted
Verification Attempted	Z3/verification path returns a trust-state result	Accepted or Rejected
Accepted	Policy profile permits output for returned state	Emitted
Rejected	No acceptable candidate or disallowed trust state	Terminal Failure
Emitted	Target dialect artifact generated successfully	Terminal Success

This state-oriented design keeps control flow explicit, simplifies diagnostics, and helps maintain consistency across single-file and batch execution paths.

Chapter 5

IMPLEMENTATION

5.1 Pseudocode

Algorithm 1 LoopHole lifting workflow

Require: Input MLIR file F , target dialect T , policy profile P

Ensure: Lifted MLIR artifact and trust-state report

```
1: Read MLIR text from  $F$ 
2: Parse loop structure, access patterns, and arithmetic operations
3: if parsing fails then
4:   return REFUTED with diagnostic
5: end if
6: Generate candidate sketches from library using structural filters
7: Score candidates using symbolic tracing
8: Select top- $k$  candidates by confidence
9: for each candidate sketch  $s$  in top- $k$  do
10:   Run verification check using Z3-backed evaluator
11:   if result is PROVED then
12:     Emit lifted MLIR for sketch  $s$  in target  $T$ 
13:     return emitted artifact with state PROVED
14:   else if result is UNPROVED_TIMEOUT and profile  $P$  is exploratory then
15:     Emit lifted MLIR as partial exploratory output
16:     return emitted artifact with state UNPROVED_TIMEOUT
17:   end if
18: end for
19: return REFUTED with best-candidate diagnostics
```

5.2 Implementation Details

The implementation follows the staged design described earlier and is organized as a Python package under `POC_initial_demo/src/loophole`. Each stage is isolated as a module and coordinated by a central orchestration layer. This structure allows targeted hardening of parser, verifier, or emitter

behavior without changing the full pipeline.

Module mapping

Table 5.1: Implemented modules and their roles

Module	Primary Function	Key Output
<code>affine_extractor.py</code>	Deterministic token-scanning parser for Affine/scf loops	LoopNestInfo data model with bounds, reads/writes, and compute ops
<code>sketch_library.py</code>	Operation sketch definitions for target dialects	Canonical sketch catalog used by matcher and verifier
<code>sympy_tracer.py</code>	Symbolic expression tracing and confidence scoring	Ranked candidates with confidence values
<code>z3_checker.py</code>	Formal equivalence check and trust-state assignment	Verification report (PROVED/UNPROVED_TIMEOUT/REFUTED) [8]
<code>emitter.py</code>	Target-dialect text emission for accepted sketches	Lifted Linalg/StableHLO MLIR artifact [7]
<code>lifter.py</code>	End-to-end orchestration and policy control	Final lift result object and diagnostics
<code>cli.py</code>	User-facing command interface	Single-file, verify, batch, and demo command surfaces

Workflow implementation

The implemented command workflows used during validation are summarized below.

Table 5.2: Operational command workflows used in the project

Workflow	Representative Command	Observed Purpose
Single lift	<code>loophole lift <input>.mlir --target linalg</code>	Generate one lifted artifact with trust-state output
Verification-first run	<code>loophole verify <input>.mlir --sketch <name></code>	Evaluate equivalence behavior and diagnostics
Batch benchmark	<code>loophole batch tests/fixtures --report</code>	Process canonical fixtures and produce structured JSON report
Demo health check	<code>loophole demo --target linalg</code>	Quick pipeline sanity check on bundled fixtures
Compile-then-lift	Docker/Polygeist assisted flow	Convert C/C++ source to Affine IR before lifting

Verification and acceptance behavior

The current implementation distinguishes trusted and exploratory outcomes through explicit trust states. `PROVED` outputs represent successful equivalence checks; `UNPROVED_TIMEOUT` indicates unresolved proof attempts under time/resource constraints; `REFUTED` indicates mismatch or failure conditions. This state model allows downstream users to separate strict acceptance from exploratory analysis without hiding uncertainty.

Reporting and reproducibility

Batch execution writes structured JSON outputs that capture fixture selection, profile metadata, timing summaries, and per-file outcomes. Combined with the Dockerized toolchain path, this supports reproducible weekly benchmarking and regression analysis across project iterations.

Chapter 6

RESULTS

6.1 Result Snapshots

The project outputs include trust-state summaries, lifted MLIR artifacts, and machine-readable benchmark reports. As requested, this section keeps placeholder images for final screenshots while presenting complete analysis text and quantitative tables.



Figure 6.1: Placeholder for strict-profile batch summary output.

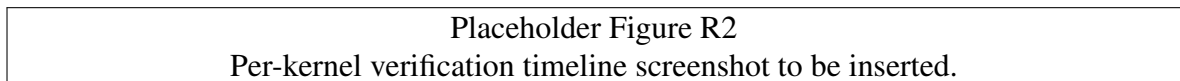


Figure 6.2: Placeholder for per-kernel verification timeline view.



Figure 6.3: Placeholder for source-vs-lifted artifact visualization.

6.2 Comparative Analysis

Table 6.1 summarizes how LoopHole currently compares with representative systems discussed in the survey. The comparison highlights that the project has a practical and reproducible pipeline in place, while larger research-grade challenges remain open in coverage and generalized proof depth.

Table 6.1: Comparative positioning against key lifting systems

System	Primary Strategy	Reported Strength	Current Constraint
mlirSynth [3]	Enumerative synthesis in MLIR dialect space	Strong TPU speedup potential	Search complexity increases rapidly with longer operation chains
Tenspiler [4]	SMT-verified lifting pipeline	Formal verification orientation	Hard cases encounter invariant/solver complexity limits
Tensorize [5]	Symbolic tracing and algebraic sketch solving	High dense-kernel benchmark performance	Focused on dense/static settings
STAGG [6]	LLM-guided probabilistic grammar with guided search	High benchmark solve rates and fast synthesis	Dense-focus leaves open sparse/dynamic fronts
LoopHole (this work)	Deterministic extraction + symbolic ranking + trust-state verification	Reproducible end-to-end implementation with strict-profile reporting	StableHLO parity and broader weak-kernel proof envelope are still being hardened

6.3 Performance Analysis

A strict-profile batch report over seven canonical fixture files shows fully accepted outputs for the evaluated set. The run captured deterministic metadata, per-file trust state, and verification latency distribution.

Table 6.2: Strict-profile batch summary (fixture set size = 7)

Metric	Observed Value	Notes
Total processed fixtures	7	Canonical fixture set in batch report
PROVED count	7	All processed files accepted in strict profile
UNPROVED_TIMEOUT count	0	No timeout-state outputs in this run
REFUTED count	0	No refuted outputs in this strict batch snapshot
Average verification latency	12.87 ms	Mean over per-file elapsed times
Latency range	5.27 ms to 33.98 ms	Minimum and maximum observed values

Table 6.3: Per-kernel verification latency from strict-profile report

Kernel Fixture	State	Elapsed (ms)	Matched Sketch
conv1d.mlir	PROVED	33.98	linalg.conv_1d_ncw_fcw
conv2d.mlir	PROVED	18.11	linalg.conv_2d
matmul.mlir	PROVED	11.91	linalg.matmul
dot_product.mlir	PROVED	8.39	linalg.dot
reduce_sum.mlir	PROVED	6.54	linalg.reduce(addf)_rowsum
elementwise_add.mlir	PROVED	5.91	linalg.map(addf)
transpose.mlir	PROVED	5.27	linalg.transpose

6.3.1 Applications

Based on the current implementation envelope, practical application areas include:

- Legacy dense linear-algebra kernels requiring migration into MLIR tensor workflows.
- Compiler research experimentation where trust-state output is required per transformation.
- Education and prototyping for verification-aware lifting in MLIR toolchains.
- CI-oriented regression checks on canonical loop-kernel fixture suites.

6.3.2 Limitations

The present results should be interpreted with scope awareness:

- The strongest outcomes are currently observed on constrained affine dense kernels.
- StableHLO coverage has improved but is not yet full parity with the mature Linalg path.
- Dynamic-shape universal proof strategies and sparse tensor lifting remain future research tracks.
- Broader real-world corpus validation is still an ongoing hardening objective.

Chapter 7

CONCLUSION & FUTURE WORK

7.1 Conclusion

This project demonstrates that automated lifting from loop-level MLIR Affine programs to higher-level tensor dialects is practically achievable with a verification-aware architecture. The implemented pipeline integrates deterministic parsing, sketch-guided symbolic matching, Z3-backed verification signals, and target-dialect emission through reproducible CLI workflows. The generated outputs and strict-profile batch evidence show that the current system can reliably process a canonical dense-kernel fixture set while preserving transparent trust-state reporting.

Beyond implementation completeness, the project contributes an engineering baseline that is directly usable for experimentation and incremental hardening. The modular structure enables independent improvement of parser, verifier, and emitter layers, and the reporting workflow supports measurable progress tracking. Overall, the work establishes a solid platform for moving from prototype capability toward stronger reliability and broader kernel coverage.

7.2 Future work

Future enhancements are planned in a prioritized manner to preserve trust while expanding capability:

1. Make strict acceptance policy the default for all trusted CI and release profiles.
2. Improve weak-kernel verification reliability, especially for more complex convolution and reduction variants.
3. Expand StableHLO parity with improved diagnostics and no silent fallback behavior.

4. Increase robustness on real frontend-generated MLIR corpora with explicit unsupported-pattern reporting.
5. Strengthen reproducible benchmark dashboards for weekly trend and regression visibility.
6. Prepare the next research track from the identified open directions: sparse lifting, transform-dialect schedule synthesis, and dynamic-shape reasoning.

These steps align the project with both immediate engineering quality goals and long-term research novelty opportunities.

References

- [1] C. Lattner, M. Amini, U. Bondhugula, and others, "MLIR: Scaling Compiler Infrastructure for Domain Specific Computation," in *Proceedings of CGO*, 2021.
- [2] W. S. Moses, L. Chelini, R. Zhao, and O. Zinenko, "Polygeist: Raising C to Polyhedral MLIR," in *Proceedings of PACT*, 2021.
- [3] A. Brauckmann, E. Polgreen, T. Grosser, and M. F. P. O'Boyle, "mlirSynth: Automatic, Retargetable Program Raising in Multi-Level IR using Program Synthesis," in *Proceedings of PACT*, 2023.
- [4] J. Qiu, C. Cai, S. Bhatia, N. Hasabnis, P. Koanantakool, and A. Cheung, "Tenspiller: A Verified-Lifting-Based Compiler for Tensor Operations," in *Proceedings of ECOOP*, 2024.
- [5] R. Fu, J. Wickerson, N. Vasilache, and T. Grosser, "From Loops to Tensors: Lifting C Loops to High-Level Tensor Computations," in *Proceedings of CGO*, 2025.
- [6] S. Bhatia, J. Qiu, S. A. Seshia, and A. Cheung, "STAGG: LLM-Based Synthesis of Tensor Algebra with Generalization Guarantees," in *Proceedings of PLDI*, 2025.
- [7] OpenXLA Project, "StableHLO Specification," Online. Available: <https://openxla.org/stablehlo>. Accessed: Apr. 26, 2026.
- [8] L. de Moura and N. Bjørner, "Z3: An Efficient SMT Solver," in *Proceedings of TACAS*, 2008.
- [9] OpenXLA Project, "XLA: Optimizing Compiler for Machine Learning," Online. Available: <https://github.com/openxla/xla>. Accessed: Apr. 26, 2026.
- [10] IREE Authors, "IREE: Intermediate Representation Execution Environment," Online. Available: <https://iree.dev/>. Accessed: Apr. 26, 2026.